EXPERIMENT 6

Hamming Code for Error Detection and Correction

AIM

To develop a program that applies the Hamming Code method for detecting and correcting errors, and to test its ability to correct an error in transmitted data.

ALGORITHM

Sender

1. Accept the text message as input.
2. Represent each character using its 8-bit binary value.
3. Split the binary sequence into groups of four bits.
4. Insert parity bits with the Hamming (7,4) coding method.
5. Write the encoded sequence into the file channel.txt.

Receiver

1. Load the encoded sequence from channel.txt.
2. Separate the sequence into 7-bit Hamming codewords.
3. Compute the parity-check values.
4. Use the syndrome value to identify the bit containing an error.
5. Flip the incorrect bit to correct it.
6. Discard the extra parity bits.
7. Convert the remaining binary sequence into ASCII characters.
8. Show the recovered message.

PROGRAM

Sender Program

```
def hamming_encode(data):
    result = ""

    for i in range(0, len(data), 4):
        d = data[i:i+4]

        while len(d) < 4:
            d += "0"

        d1, d2, d3, d4 = map(int, d)

        # Hamming (7,4) parity bits
        p1 = d1 ^ d2 ^ d4
        p2 = d1 ^ d3 ^ d4
        p4 = d2 ^ d3 ^ d4

        code = [p1, p2, d1, p4, d2, d3, d4]
        result += ''.join(map(str, code))

    return result


text = input("Enter text: ")
binary = ''.join(format(ord(ch), '08b') for ch in text)
encoded = hamming_encode(binary)

with open("channel.txt", "w") as file:
    file.write(encoded)

print("Binary Data :", binary)
print("Hamming Code:", encoded)
```

```
print("Data saved in channel.txt")
```

Receiver Program

```
def hamming_decode(code):
    data = ""

    for i in range(0, len(code), 7):
        block = list(map(int, code[i:i+7]))

        if len(block) < 7:
            continue

        # Calculate syndrome
        s1 = block[0] ^ block[2] ^ block[4] ^ block[6]
        s2 = block[1] ^ block[2] ^ block[5] ^ block[6]
        s4 = block[3] ^ block[4] ^ block[5] ^ block[6]

        error_position = s1 + (s2 * 2) + (s4 * 4)

        if error_position != 0:
            print("Error found at position:", error_position)
            block[error_position - 1] ^= 1
            print("Error has been corrected.")

        # Remove parity bits and retain data bits
        data += str(block[2])
        data += str(block[4])
        data += str(block[5])
        data += str(block[6])

    return data


with open("channel.txt", "r") as file:
    encoded = file.read().strip()

binary = hamming_decode(encoded)
text = ""

for i in range(0, len(binary), 8):
    byte = binary[i:i+8]

    if len(byte) == 8:
        text += chr(int(byte, 2))

print("Received Data:", text)
```

TEST RUN

Input: HELLO

Sender Output:

```
Binary Data : 01001000010001010100110001001100010010011111
Hamming Code: <encoded binary data>
Data saved in channel.txt
```

Receiver Output:

```
Received Data: HELLO
```

When a single bit in channel.txt is manually altered, the receiver identifies the error and corrects it:

```
Error detected at position: 5
Error corrected.
Received Data: HELLO
```

RESULT

Hence, the Hamming Code method was implemented successfully to detect and correct a single-bit error in the transmitted message.